

VERİ YAPILARI

Dr. Öğr. Üyesi Fatma AKALIN

Veri Yapısı, bilgisayarda **verinin saklanması** (tutulması) ve **organizasyonu** için bir yoldur. Veri yapılarını saklarken ya da organize ederken **iki şekilde** çalışacağız;

- 1- Matematiksel ve mantıksal modeller (Soyut Veri Tipleri – Abstract Data Types - ADT)
- 2- Uygulama (Implementation)

Matematiksel ve mantıksal modeller (Soyut Veri Tipleri – Abstract Data Types - ADT)

Bir veri yapısına bakarken tepeden (yukarıdan) **soyut** olarak ele alacağız. Yani hangi işlemlere ve özelliklere sahip olduğunu irdeleyeceğiz. Örneğin bir **TV alıcısına** soyut biçimde bakarsak **elektriksel bir alet olduğu** için onun açma ve kapama **tuşu**, sinyal almak için bir **anteni** olduğunu göreceğiz. Aynı zamanda **görüntü** ve **ses** vardır.

Fakat bu TV'ye **matematiksel ve mantıksal modelleme açısından** bakarsak içindeki devrelerin ne olduğu veya nasıl tasarlandıkları konusuyla ve hangi firma üretmiş gibi bilgilerle **ilgileneceğiz**. Bu bakışın içerisinde uygulama (implementasyon) yoktur

Uygulama (Implementation)

Matematiksel ve mantıksal modellemenin uygulamasıdır. Ancak, uygulama aşamasında bazı kısıtlamalar ortaya çıkar. Örneğin **diziler**, programlamada çok kullanışlı veri yapıları olmasına rağmen bazı **dezavantajları** ve **kısıtları** vardır. Bunlar;

- Derleme aşamasında dizinin boyutu bilinmelidir,

- Diziler bellekte sürekli olarak yer kaplarlar. Örneğin int türden bir dizi tanımlanmışa, eleman sayısı çarpı int türünün kapladığı alan kadar bellekte yer kaplayacaktır,

- Ekleme işleminde dizinin diğer elemanlarını kaydırmak gerekir. Bu işlemi yaparken dizinin boyutunun da aşılmaması gerekir,

- Silme işlemlerinde değeri boş elemanlar oluşacaktır

1. Derleme Aşamasında Dizinin Boyutu Bilinmelidir

```
int dizi[5];
```

```
+---+---+---+---+---+
```

```
| 0 | 0 | 0 | 0 | 0 |
```

```
+---+---+---+---+---+
```

UYARI

Bu dizinin boyutu
önceden belirlenmiştir ve
çalışma zamanında
geniştirilemez.

2. Diziler Bellekte Sürekli Olarak Yer Kaplarlar

```
int dizi[5] = {10, 20, 30, 40, 50};
```

```
+-----+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+-----+
```

UYARI

Bellek adresleri de
sıralı olacak şekilde
tahsis edilir.

3. Ekleme İşleminde Dizinin Diğer Elemanlarını Kaydırmak Gerekir

`int dizi[5] = {10, 20, 30, 40}` dizisinin 15 değerini 2. indekse eklemek istiyoruz.

```
+---+---+---+---+---+
| 10 | 20 | 30 | 40 |   |
+---+---+---+---+---+
```

UYARI: 15 eklenirken 30 ve 40
sağa kaydırılır.

Eğer dizinin boyutu doluyorsa
yeni eleman eklemek mümkün
değildir.

```
+---+---+---+---+---+
| 10 | 20 | 15 | 30 | 40 |
+---+---+---+---+---+
```

4. Silme İşleminde Değeri Boş Elemanlar Oluşur

`int dizi[5] = {10, 20, 30, 40, 50}`
dizisinde 20 değerini silelim.

```
+----+----+----+----+----+
| 10 | 20 | 30 | 40 | 50 |
```

Silme işleminden sonra:

```
+----+----+----+----+----+
| 10 |  | 30 | 40 | 50 |
+----+----+----+----+----+
```

Bu durumda, boş eleman kalır.
Alternatif olarak tüm elemanları
kaydırabiliriz(manuel):

```
+----+----+----+----+----+
| 10 | 30 | 40 | 50 |  |
+----+----+----+----+----+
```

Bu işlemde tüm elemanlar
kaydırıldığı için $O(n)$ zaman
karmaşıklığı oluşur.

Sonuç:

Dizilerin boyutu sabittir ve değiştirilemez.

Bellekte bitişik olarak saklanırlar.

Ekleme ve silme işlemleri zaman alıcıdır.

Bağlı listeler (Linked List) gibi dinamik veri yapıları, dizilerin bu dezavantajlarını aşmak için geliştirilmiştir.

Dr. Öğr. Üyesi Fatma AKALIN

Uygulama kapsamında bu ve bunun gibi sorunların üstesinden gelmek
Bağlı Listelerle (Linked List) çözülebilmek potansiyeline sahiptir.

Bu bağlamda bu ders kapsamında inceleyeceğimiz bazı veri tipleri;

- Arrays (Diziler)
- Linked List (Bağlı liste)
- Stack (Yığın)
- Queue (Kuyruk)
- Tree (Ağaç)
- Graph (Graf, çizge)

Veri yapılarını çalışırken,

- 1- Mantıksal görünüşüne bakacağız,
- 2- İçinde barındırdığı işlemlere bakacağız.

DİZİ VERİ YAPISI

Diziler **art arda** gelen, **belirli** bir **sayıda**, **aynı tip** bilgiyi saklayan bellek elemanlarıdır. Dizi veri yapısının **sonlu sayıda** elemandan oluşması ve bu elemanların **bellekte fiziksel olarak ard arda** gelmesi, **aynı tip** verilerden oluşuyor olması, bu veri yapısını özel kılmakta ve bu **veri yapısının elemanlarına bir çırpıda** ulaşabilmeyi sağlamaktadır. Dizi veri yapısı sayesinde aynı anda **bir değişken ismi altında birden fazla bilgi** tutmakta mümkündür. Diziler, bilginin bellekte saklanma düzeninden programcıyı **soyutlayabildiğinden** programların anlaşılabilirliğini ve **okunabilirliğini** arttırır ve programlamaya esneklik sağlar.

Program içerisinde **aynı tipte** ve **çok sayıda** bilginin mevcut olması durumunda **her bilgi için ayrı ayrı değişken** kullanılması pratik **değildir**. Çünkü **her değişkene** isim vermek, bilgi aktarmak, tip ve bellek bilgisi sağlamak ve bu bilgiler üzerinde işlem yapmak veya bu bilgileri görüntülemek istendiğinde bu işlerin her biri için ayrı ayrı komut kullanılması **karmaşıklığı** artıracak ve programı içinden çıkılmaz bir hale getirecektir. Bu da programcılık da istenmeyen bir durumdur ve **bütün programlama dilleri için de geçerlidir**.

Örneğin **100 adet isim** ve **Adres** bilgisine **aynı anda hafıza** hafızada tutmak için **100 adet isim ve 100 adet adres** olmak üzere **toplam 200 adet değişkene** ihtiyaç vardır. Dizi kullanılması durumunda ise her biri **100 elemanlı iki dizi değişken adı** kullanılması **yeterli** olacaktır.

Genel olarak bilgisayarda işlenen **verilerin sayısı fazladır**. Çok sayıda verinin **bilgisayara girilmesi/aktarılması, hızlı ve doğru şekilde işlenebilmesi, uygun şekilde saklanabilmesi için** belirli bir düzende olması gerekmektedir. Belirli bir düzende olan verilerin işlenmesi hem daha **kolay** hem daha **pratiktir**. Bu nedenle bilgisayar programlarında; çoklu verileri işlemek için **dizi** olarak adlandırılan **sıralı veri/bilgi alanları** kullanılır. Programlarda **tek isimle/ tanımlayıcı ile adlandırılan bu veri alanları dizi ismi yanındaki indis numarası ile çağırılıp istenirler**. En temel çoklu veri alanı olan diziler **3 başlık** altında incelenebilir.

Bir boyutlu diziler: Vektör olarak da adlandırılan ve sadece satır veya sütundan, yani **tek sıradan oluşan** ardışık veri alanıdır.

İki boyutlu diziler: Matris olarak da adlandırılan ve **satır ile sütundan** oluşan veri alanıdır.

Çok boyutlu diziler: **3 veya daha fazla boyutlu veri alanlarıdır.** Örneğin uzay, **genişlik-uzunluk-yükseklikten** oluşan üç boyutlu veri alanıdır.

İster **bir** ister **iki** boyutlu olsun, veriler belleğe **ardışık** olarak yerleşmektedirler.

1. Bir Boyutlu Dizi (Vektör)

ÖRNEK --- > `int vektor[5] = {10, 20, 30, 40, 50};`

Bellekteki Yerleşimi

+---+---+---+---+---+

| 10 | 20 | 30 | 40 | 50 |

+---+---+---+---+---+

2. İki Boyutlu Dizi (Matris)

ÖRNEK --- > `int matris[2][3] = { {1, 2, 3}, {4, 5, 6} };`

Mantıksal Görünümü---> 1 2 3

4 5 6

Bellekteki Yerleşimi (Row-major Order):

+---+---+---+---+---+---+

| 1 | 2 | 3 | 4 | 5 | 6 |

+---+---+---+---+---+---+

Tüm veriler ardışık olarak tutulur.

3. Üç Boyutlu Dizi (Çok Boyutlu)

Üçüncü boyut, farklı **katmanlar (layers)** içerir.

ÖRNEK --- > `int uzay[2][2][3] = { { {1, 2, 3}, {4, 5, 6} }, { {7, 8, 9}, {10, 11, 12} } };`

Mantıksal Görünüm:

Katman 1: 1 2 3

4 5 6

Katman 2: 7 8 9

10 11 12

Bellekteki Yerleşimi (Row-major Order):

```
+---+---+---+---+---+---+---+---+---+---+---+---+
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
+---+---+---+---+---+---+---+---+---+---+---+---
```

Veriler **katmanlar halinde** tutulsa da bellekte **sıralı** yerleştirilir.

1D Dizi (Vektör) → Tek sıra halinde ardışık saklanır.

2D Dizi (Matris) → Satırlar birleştirilerek ardışık saklanır.

3D Dizi (Çok Boyutlu) → Katmanlar sıralı şekilde bellekte saklanır.

BİR BOYUTLU DİZİ UYGULAMALARI

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int N;
5  int main() {
6      cout<<"Dizinin eleman sayisini giriniz"<<endl;
7      cin>>N;
8      int A[N];
9      for(int i=0; i<N; i++){
10         cout<<"A ("<<i+1<<" )=";
11         cin>>A[i];
12     }
13     cout<<"Dizinin tum elemanlari:"<<endl;
14     for(int i=0; i<N; i++){
15         cout<<"A ("<<i+1<<" )="<<A[i]<<endl;
16     }
17     return 0;
18 }
```

p\VERİ YAPILARI\diziler>g++ -o main main.cpp

p\VERİ YAPILARI\diziler>main
ni giriniz

i:

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\diziler>g++ -o main2 main2.cpp
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\diziler>main2
```

Cumleyi giriniz...Dr. Öğr. Üyesi Fatma Akalın

Cumlede 5tane kelime var

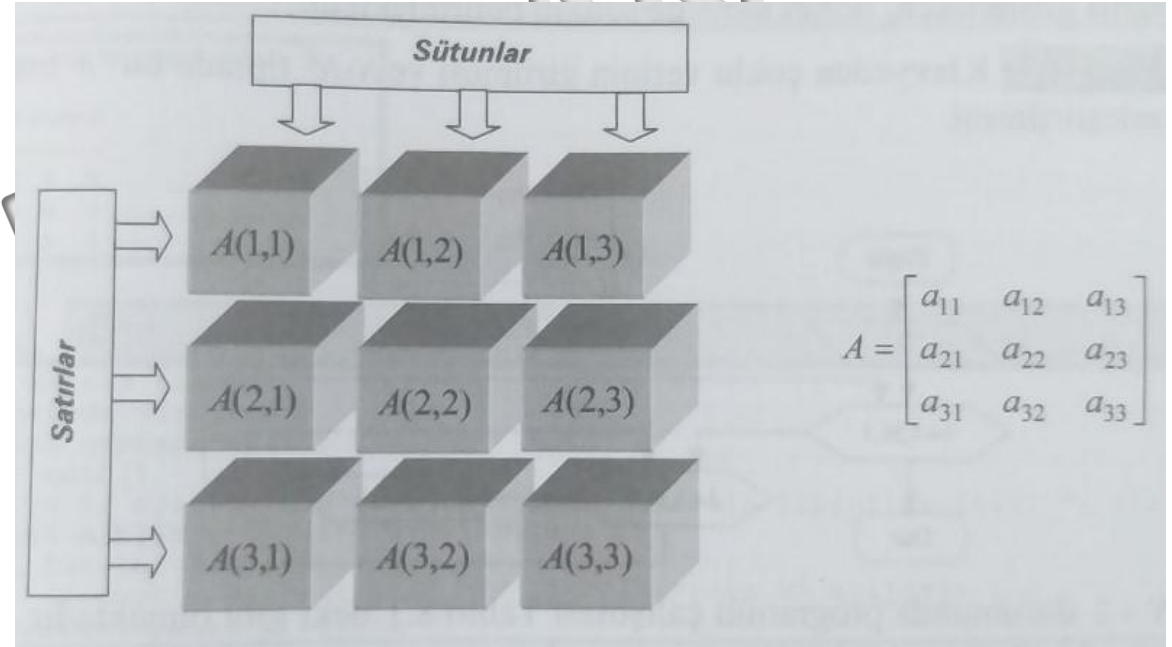
```
#include <iostream>
using namespace std;

string c,is=" ";
int i,j,say=0;

int main() {
    cout<<"Cumleyi giriniz...";
    getline(cin,c);
    for(i=0; i<c.size(); i++){
        for(j=0; j<is.size(); j++){
            if(c[i]==is[j]){
                say++;
            }
        }
    }
    cout<<endl<<"Cumlede " <<(say+1)<<"tane kelime var"<<endl;
    return 0;
}
```

İKİ BOYUTLU DİZİ UYGULAMALARI

Mühendislik uygulamalarında özellikle de **çoklu verilerle** ilgili olan işlemlerde yoğun olarak kullanılan yapılardan birisi de **matrislerdir**. Tablo gibi olan **iki boyutlu diziler** **matris** olarak isimlendirilir.



```

1  #include <iostream>
2
3  using namespace std;
4
5  int i,j,N;
6
7  int main() {
8      cout<<"Kare matrisin tipini giriniz";
9      cin>>N;
10     int A[N][N],B[N][N],C[N][N];
11     cout<<endl<<"A matrisi"<<endl;
12     for(i=0; i<N; i++){
13         for(j=0; j<N; j++){
14             cin>>A[i][j];
15         }
16     }
17     cout<<endl<<"B matrisi"<<endl;
18     for(i=0; i<N; i++){
19         for(j=0; j<N; j++){
20             cin>>B[i][j];
21         }
22     }
23     cout<<endl<<"A+B matrisi"<<endl;
24     for(i=0; i<N; i++){
25         for(j=0; j<N; j++){
26             C[i][j]=A[i][j]+B[i][j];
27             cout<<C[i][j]<<"\t";
28         }
29         cout<<endl;
30     }
31     return 0;
32 }

```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\diziler>g++ -o main3 main3.cpp
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\diziler>main3
```

```
Kare matrisin tipini giriniz2
```

```
A matrisi
```

```
1
1
1
1
1
```

```
B matrisi
```

```
2
2
2
2
2
```

```
A+B matrisi
```

```
3      3
3      3
```


Dizi ile Baęlı Liste Arasındaki Farklar

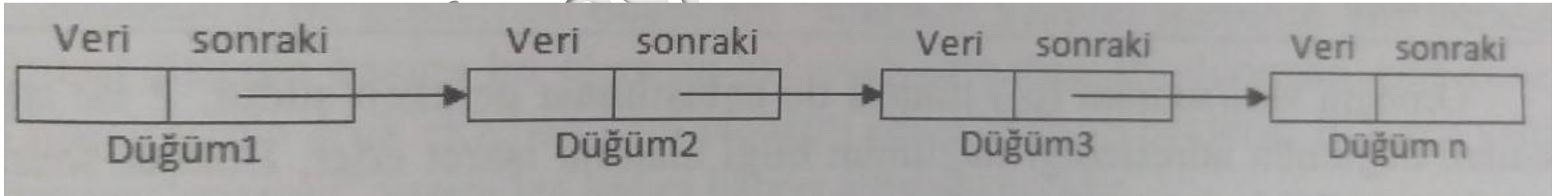
Dizi (Array)	Baęlı Liste (Linked List)
Dizilerin belirli bir kapasitesi vardır. (Program başladıęında belirlenir)	Baęlı listelerin bir kapasitesi yoktur belleęin izin verdięi miktarda büyüeyebilirler.
Diziler birbirinin arkasına sıralı bir şekilde gelir. (Dizi[0], Dizi[1] den 4 bayt (integer dizi için) sonradır)	Baęlı listedeki elemanlar bellekte herhangi bir yerde olabilirler.
Bir diziye eleman eklemek ve çıkartmak çok maliyetlidir. (5 elemanlık bir diziye 6. elemanı eklemek isterseniz ilk önce önceki 5 elemanlık diziyi bellekte başka bir yere taşımamız gerekir)	Baęlı listelerde eleman eklemek ve çıkarmak nispeten daha kolaydır. (Sadece en son düęümü veya silinecek düęümü bulmak gerekir)
Dizilerde arama çok daha kolaydır. Çünkü bellekte ardışık yer kaplarlar. (Algoritma Karmaşıklığı $O(1)$)	Baęlı listelerde arama yapmak zordur çünkü bir düęüme ulaşmak için önceki düęümlerden gelmeniz gerekir. (Algoritma Karmaşıklığı $O(n)$)
Diziler bellekte daha az yer kaplar (Sadece değerleri tutarlar)	Baęlı listeler bellekte daha fazla yer kaplar (Hem değerleri hem de sonraki düęümün adresini tutarlar)

<https://www.ysancar.com/veri-yapilari/dizi-ile-bagli-liste-arasindaki-farklar/>

BAĞLI LİSTE

Bağlantılı liste, adı üzerinde **aynı kümeye ait veri parçalarının** birbirlerine bellek üzerinde **sanal olarak bağlanması** ile oluşturulur. Tüm veri bir tren katarı gibi birbirine sanal bağlı parçalardan oluşur. Dolayısıyla bağlantılı liste veri yapısında birisi **veri bloğu** diğeri **bağlantı bilgisi** olmak üzere temelde iki kısım bulunur. **Veri** kısmında o uygulama için **gerekli bir veya birkaç bilgi** bulunabilir. **İşaretçi** olarak adlandırılan bağlantı kısmında ise bağlantının nereye yapıldığını gösteren bir veya birkaç **adres** bilgisi bulunabilir. **Bağlantılı liste, liste veri modelinin uygulanma şekillerinden biridir** denilebilir.

Ekleme ve silme işlemlerinin **kolayca** yapılabilmesi, **aradan** bir kayıt **silindiğinde** veya **yeni bir kayıt** eklendiğinde **diğer kayıtlara dokunulmaması**, uygulama ve probleme göre değişik liste yapılarının **kolayca tanımlanabilmesi** açılarından bağlantılı liste algoritma tasarımında önemli bir yere sahiptir.



Bağlantılı listede **en önemli anahtar** sözcük veri parçalarının bir **bağlantı bilgisi üzerinden sanal olarak birbirine bağlanmasıdır**. Genel olarak veri parçaları bellekte art arda **sırada değil** de **farklı alanlarında** bulunurlar. Dolayısıyla **bağlantılı listedeki** herhangi bir **veri** parçasına onun **başlangıç adresi veya indisi** bilinmediği sürece **doğrudan erişim yapılamaz**. Listenin başından başlanıp aynı bir tren katarındaki bir vagon dan diğerine geçilmesi gibi **veri parçaları üzerinden ilerlenmesi** gerekir. Örneğin bağlantı Listenin **10.** veri parçasına erişmek için ondan önceki **dokuz tane veri parçası üzerinden geçilmelidir**. Bağlantılı liste veri modeli **dinamik bellek kullanımı** sunan önemli veri modellerinden birisidir. Dolayısıyla **listedeki düğüm sayısı** sistemin sahip olduğu **bellekle doğru orantılı olur**

Bağlantılı liste veri modeli «bilgisayar biliminde çok önemli bir yere sahiptir» denilebilir. **Bazı problemler** bu veri modeline **dayanılarak çözülp** tasarlanırken bazı uygulamalar içinde bağlantılı listeler, **ara gereksinim** olur. Örneğin verilerin belirli bir sırada tutulması, işlenmesi için sıraya konulması, aradan kolayca çıkarılması ve sıraya kolayca eklenmesi gibi işlemler gerektiren uygulamalarda seçim olur. Bağlantılı listelerin **zayıf** yönünü ise liste üzerindeki ara elemanlara doğrudan erişilmemesi ve sık sık arama gerektiren uygulamalarda arama karmaşıklığının fazla olmasıdır. **Eğer ara elemanlara doğrudan hızlı bir şekilde erişilmesi bekleniyorsa dizi, aramanın en az karmaşıklıkla yapılması bekleniyorsa sıralı ağaç modelleri kullanılmalıdır.**

Yazılan uygulamalarında bağlantılı liste veri modeli ara bir araç olarak oldukça fazla kullanılır. Çünkü doğası gereği **birçok uygulama bağlantılı liste veri modeline yatkın** olmaktadır

Bağlantılı Liste Kavramları

Bağlantılı liste «temelde liste veri modelinin bir uygulamasıdır» denilebilir.

Burada yapılan liste elemanlarının bellekte tutulma şekli ve elemanların liste özelliğini bozmadan aynı kümeye ait olduklarını gösteren bağlantıların yapılmasıdır. Bağlantılı bir liste Yalın olarak yandaki şekilde gösterildiği gibi tasvir edilebilir

- Her kutu bir **node (düğüm)**
- Node yapısı genelde şöyle olur:

```
struct Node {  
    int veri;  
    struct Node* next;  
};
```

ilk



[Ali | 0x200] → [Top | 0x350] → [Koş | NULL]
0x100 0x200 0x350

Bağlantı kısmı, teknik olarak pointer tutuyor

✓ Pointer'ın içinde ise adres (memory address) var

Yani: next bir pointer değişkendir

Bu pointer, bir sonraki düğümün adresini saklar

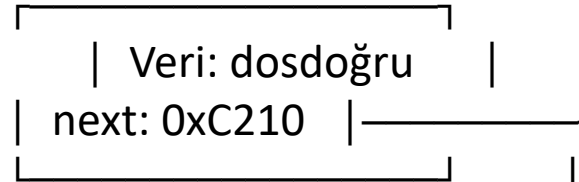
ilk



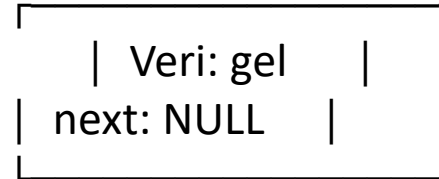
0xF450



0xB330



0xC210



son

Bağlantılı listenin veri yapısı bir sonraki örnekte de görüleceği gibi birisi **veri bloğu** diğeri **bağ** olarak adlandırılan **iki ayrı parçadan** oluşur. Veri bloğu uygulamanın gereksinimine göre bir karakterlik olabileceği gibi bir string (sözce), bir tam sayı, bir kesirli sayı veya bunların karışımından oluşan birden çok veriden meydana gelebilir. Örneğin **öğrenci otomasyonu sisteminde** bir öğrencinin tüm bilgilerini içeren ve birçok sözce ve sayıdan oluşan bir veri kümesi de olabilir. **Bağ kısmı** ise bağlantılı liste üzerinde bir **sonraki veri parçasının yerine işaret eden** bir **adres** veya **indis** bilgisidir. Bir sonraki örnek; tek yönlü olup liste üzerinde baştan sona doğru bir yönde hareket edilebilir.


```
1 #include <iostream>
2 using namespace std;
3
4 struct node{
5     string kullaniciadi;
6     int parola;
7     node * next;
8 };
9
10 int main() {
11     struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giris yapacak
12     struct node * uye2=new node();
13     struct node * uye3=new node();
14     struct node * uye4=new node();
15     struct node * uye5=new node();
16
17     uye1->kullaniciadi="a1";
18     uye2->kullaniciadi="a2";
19     uye3->kullaniciadi="a3";
20     uye4->kullaniciadi="a4";
21
22     uye1->parola=111;
23     uye2->parola=222;
24     uye3->parola=333;
25     uye4->parola=444;
26
27     uye1->next=uye2;
28     uye2->next=uye3;
29     uye3->next=uye4;
30     uye4->next=NULL;
31
32     cout<<"1. dugumun kullanıcı adı ve şifresi : "<<uye1->kullaniciadi<<" & "<<uye1->parola<<endl;
33     cout<<"2. dugumun kullanıcı adı ve şifresi : "<<uye2->kullaniciadi<<" & "<<uye2->parola<<endl;
34     cout<<"3. dugumun kullanıcı adı ve şifresi : "<<uye3->kullaniciadi<<" & "<<uye3->parola<<endl;
35     cout<<"4. dugumun kullanıcı adı ve şifresi : "<<uye4->kullaniciadi<<" & "<<uye4->parola<<endl;
36
37     return 0;
38
39
40 }
```

KALIN

```
D:\DRS\kodlar\kitaplar\kitap\tekyonlubagliliste>g++ -o ornek1 ornek1.cpp
```

```
D:\DRS\kodlar\kitaplar\kitap\tekyonlubagliliste>ornek1
```

```
1. dugumun kullanıcı adı ve şifresi : a1 & 111
2. dugumun kullanıcı adı ve şifresi : a2 & 222
3. dugumun kullanıcı adı ve şifresi : a3 & 333
4. dugumun kullanıcı adı ve şifresi : a4 & 444
```

Bağlı listenin
elemanlarını
while döngüsü
ile yazdırma

Dr. Öğr.

```
cpp x ornek2.cpp x ornek3.cpp x ornek4.cpp x ornek5.cpp x
#include <iostream>
using namespace std;

struct node{
    string kullanicadi;
    int parola;
    node * next;
};

int main() {
    struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giris yapacak
    struct node * uye2=new node();
    struct node * uye3=new node();
    struct node * uye4=new node();
    struct node * uye5=new node();

    uye1->kullanicadi="a1";
    uye2->kullanicadi="a2";
    uye3->kullanicadi="a3";
    uye4->kullanicadi="a4";

    uye1->parola=111;
    uye2->parola=222;
    uye3->parola=333;
    uye4->parola=444;

    uye1->next=uye2;
    uye2->next=uye3;
    uye3->next=uye4;
    uye4->next=NULL;

    struct node *temp=uye1;
    int i=1;
    while (temp!=NULL) {
        cout<<i<<". dugumun kullanici adi ve sifresi : "<<temp->kullanicadi<< " & "<<temp->parola<<endl;
        temp=temp->next;
        i++;
    }
    return 0;
}
```

```
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>g++ -o ornek2 ornek2.cpp

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>ornek2
1. dugumun kullanici adi ve sifresi :a1 & 111
2. dugumun kullanici adi ve sifresi :a2 & 222
3. dugumun kullanici adi ve sifresi :a3 & 333
4. dugumun kullanici adi ve sifresi :a4 & 444
```


Bu örnekte **bağlantılı listenin başı uye1 adlı bir değişken tarafından tutulur.** Liste üzerinde işlem yapılırken **başlangıç noktası bu değişkendir.** Liste üzerindeki **ara düğümlere doğrudan erişilemez, hep** bu uye1 adlı değişken üzerinden başlanmalıdır. Örneğin üçüncü düğüme erişilmek istenirse, daha önce ilk düğünden ikincisine, ikinci düğümdende üçüncü düğümün adresi öğrenilmelidir. Ancak üçüncü düğümün **adresi elde edilmişse oradaki veriler** kullanılabilir. Bağlantılı liste üzerinde bir verinin aranması için verinin liste üzerindeki sırası kadar döngü yapılması gerekir.

En kötü durum listede olmayan bir verinin aranmasıdır. Bu durumda listenin tüm elemanlarına bakılmalıdır ve n tane çevrim yapılması gerekir. Dolayısıyla bir bağlantılı liste üzerinde arama yapmanın maliyeti en kötü durumda $O(n)$ olur. Eğer kullanılması programın çalışma maliyetini iyileştirecekse veya program tasarımını kolaylaştırıyorsa bir de ek olarak listenin son elemanını gösteren «son» adlı bir değişken kullanılır. Böyle bir değişken kullanılması özellikle **listeye ekleme yapılmasını kolaylaştırır.** Eğer **liste sırasız** özellikte ise **yani yeni gelenler listenin sonunda ekleniyorsa** doğrudan son tarafından işaret edilen elemanın arkasına eklenir. Bu durumda ekleme maliyeti $O(1)$ olacaktır. **Aksi durumda eğer son değişkeni kullanılmazsa** listenin **başından sonuna kadar dolaşılması gereken bu durumda da maliyet $O(n)$ olacaktır,** yani ekleme maliyeti artacaktır.

Aksi belirtilmediği sürece bağlantılı liste denildiğinde son kod örneğinde anlatıldığı üzere **tek yönlü bağlantılı listeler** akla gelir.

Bir liste üzerinde listeye eleman ekleme, listeden eleman çıkarma, listenin boş olup olmadığının kontrol edilmesi gibi **temel işlemler** yapılabilir. Şimdi bu işlemleri irdeleyelim.

Dr. Öğr. Üyesi Fatma AKALIN

Bağlı Listeye Eleman Ekleme

Bir bağlı listeye **her an eleman eklemek** ya da **listeden eleman çıkarmak** olası olduğundan bağlı liste veri yapısı dinamik bir veri yapısıdır. Eleman ekleme işlemini **statik veri yapısı olan dizi** üzerinde gerçekleştirdiğimizde **elemanların toplu şekilde hareketi** söz konusuydu. Fakat bağlı listede listeye bir eleman eklemek istediğimizde sadece bazı **işaretçi güncellemeleri yapılacak ve diğer elemanlara dokunulmayacaktır**

```

.cpp x ornek2.cpp x ornek3.cpp x ornek4.cpp x ornek5.cpp x
#include <iostream>
using namespace std;
struct node{
    string kullanicadi;
    int parola;
    node * next;
};
struct node * basaEkle(node * basdugum,string k_adi,int sifre){
    if(basdugum==NULL){
        struct node * dugum=new node();
        dugum->kullanicadi=k_adi;
        dugum->parola=sifre;
        dugum->next=NULL;
        basdugum=dugum;
    }
    else{
        struct node * dugum=new node();
        dugum->next=basdugum;
        dugum->kullanicadi=k_adi;
        dugum->parola=sifre;
        basdugum=dugum;
    }
    return basdugum;
}
void listedekielemanlarigetir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Listede getirilecek eleman yok";
    }
    node * temp=basdugum;
    while(temp!=NULL){
        cout<<temp->kullanicadi<<" & "<<temp->parola<<endl;
        temp=temp->next;
    }
}
int main(){
    node * root=NULL;
    root=basaEkle(root,"abas1",111);
    root=basaEkle(root,"abas2",222);
    root=basaEkle(root,"abas3",333);
    listedekielemanlarigetir(root);
    return 0;
}

```

Bağlı Listede başa eleman eklemek

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>g++ -o ornek3 ornek3.cpp
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>ornek3
```

```

abas3 & 333
abas2 & 222
abas1 & 111

```

```

1.cpp x ornek2.cpp x ornek3.cpp x ornek4.cpp x ornek5.cpp x
#include <iostream>
using namespace std;
struct node{
    string kullanıcıadi;
    int parola;
    node * next;
};

struct node * sonaEkle(node * basdugum,string k_adi,int sifre){
    if(basdugum==NULL){
        struct node * dugum=new node();
        dugum->kullanıcıadi=k_adi;
        dugum->parola=sifre;
        dugum->next=NULL;
        basdugum=dugum;
    }
    else{
        struct node * dugum=new node();
        struct node * temp=basdugum;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=dugum;
        dugum->kullanıcıadi=k_adi;
        dugum->parola=sifre;
        dugum->next=NULL;
    }
    return basdugum;
}

void listedekielemanlari getirir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Listede getirilecek eleman yok";
    }
    node * temp=basdugum;
    while(temp!=NULL){
        cout<<temp->kullanıcıadi<<" & "<<temp->parola<<endl;
        temp=temp->next;
    }
}

int main(){
    node * root=NULL;
    root=sonaEkle(root,"ason1",111);
    root=sonaEkle(root,"ason2",222);
    root=sonaEkle(root,"ason3",333);
    listedekielemanlari getirir(root);
    return 0;
}

```

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>g++ -o ornek4 ornek4.cpp
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>ornek4
ason1 & 111
ason2 & 222
ason3 & 333

```

Bağlı Listede sona eleman eklemek


```

cpp x ornek2.cpp x ornek3.cpp x ornek4.cpp x ornek5.cpp x
#include <iostream>
using namespace std;

struct node{
    string kullaniciadi;
    int parola;
    node * next;
};

struct node * ekle(node * basdugum, string k_adi, int sifre){
    if(basdugum==NULL){
        struct node * dugum=new node();
        dugum->kullaniciadi=k_adi;
        dugum->parola=sifre;
        dugum->next=NULL;
        basdugum=dugum;
    }
    else{
        struct node * dugum=new node();
        struct node * temp=basdugum;
        while(temp->next!=NULL){
            temp=temp->next;
        }
        temp->next=dugum;
        dugum->kullaniciadi=k_adi;
        dugum->parola=sifre;
        dugum->next=NULL;
    }
    return basdugum;
}

```

Bağlı Listede araya
eleman eklemek

```

//aranan degerden once hedef sayı eklenecek!!!
struct node * arayaEkle(node * basdugum, string k_adi, int sifre, int aranandeger){
    if(basdugum==NULL){
        cout<<"Liste bos";
    }
    else if(basdugum->next==NULL){
        if(basdugum->parola==aranandeger){
            struct node * dugum=new node();
            dugum->next=basdugum;
            dugum->kullaniciadi=k_adi;
            dugum->parola=sifre;
            basdugum=dugum;
        }
    }
    else{
        struct node * dugum=new node();
        struct node * temp=basdugum;
        while(temp->next->parola!=aranandeger){
            temp=temp->next;
        }
        struct node * temp2=temp->next;
        temp->next=dugum;
        dugum->kullaniciadi=k_adi;
        dugum->parola=sifre;
        dugum->next=temp2;
    }
    return basdugum;
}

void listedekielemanlarigetir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Listede getirilecek eleman yok";
    }
    node * temp=basdugum;
    while(temp!=NULL){
        cout<<temp->kullaniciadi<<" & "<<temp->parola<<endl;
        temp=temp->next;
    }
}

```

Listede aranan elemanın kesinlikle olduğunu bilerek yazdığımız kod parçası..


```

int main() {
    string k_adi;
    int sifre;
    node * root=NULL;
    cout<<"Elemanlar ekleniyor"<<endl;
    root=ekle(root,"a1",111);
    root=ekle(root,"a2",222);
    root=ekle(root,"a4",444);
    listedekielemanlarigetir(root);
    cout<<"Belirlediginiz paroladan once girilecek kullnici adini giriniz (Araya eleman ekleme)";
    cin>>k_adi;
    cout<<"Belirlediginiz paroladan once girilecek sifreyi adini giriniz (Araya eleman ekleme)";
    cin>>sifre;
    root=arayaEkle(root,"a3",333,444);
    listedekielemanlarigetir(root);
    return 0;
}

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>g++ -o ornek5 ornek5.cpp

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\tekyonlubagliliste>ornek5

Elemanlar ekleniyor

a1 & 111

a2 & 222

a4 & 444

Belirlediginiz paroladan once girilecek kullnici adini giriniz (Araya eleman ekleme)a3

Belirlediginiz paroladan once girilecek sifreyi adini giriniz (Araya eleman ekleme)333

a1 & 111

a2 & 222

a3 & 333

a4 & 444

Toplam Düğüm Sayısını Bulma Fonksiyonu

```
void toplamdugumsayisi(node * basdugum){
    if(basdugum==NULL){
        cout<<"Listede eleman yok"<<endl;
        cout<<"Toplam dugum sayisi 0";
    }
    else{
        node * iter=basdugum;
        int adet=0;
        while(iter!=NULL){
            adet++;
            iter=iter->next;
        }
        cout<<"Toplam dugum sayisi:
        "<<adet<<endl;
    }
}
```

Toplam Düğüm Sayısını bulmak için hedef metot yanda verilmiştir.

Somutlaştırılması ve kodlanması ders akışında ayrıntılı olarak yapılacaktır.

Baştan Düğüm Silme Fonksiyonu

```
ornek8.cpp x
1  #include <iostream>
2  using namespace std;
3  struct node{
4      int parola;
5      string kullaniciadi;
6      struct node * next;
7  };
8  struct node * ekle(node * basdugum, string k_adi, int sifre){
9      if(basdugum==NULL){
10         struct node * dugum=new node();
11         dugum->kullaniciadi=k_adi;
12         dugum->parola=sifre;
13         dugum->next=NULL;
14         basdugum=dugum;
15     }
16     else{
17         struct node * dugum=new node();
18         struct node * temp=basdugum;
19         while(temp->next!=NULL){
20             temp=temp->next;
21         }
22         temp->next=dugum;
23         dugum->kullaniciadi=k_adi;
24         dugum->parola=sifre;
25         dugum->next=NULL;
26     }
27     return basdugum;
28 }
29 struct node * basdugumusil(node * basdugum){
30     if(basdugum==NULL){
31         cout<<"Listede silinecek eleman yok";
32     }
33     if(basdugum->next==NULL){
34         delete basdugum;
35         basdugum=NULL;
36     }
37     else{
38         struct node * iter=basdugum;
39         struct node * temp;
40         temp=iter->next;
41         delete iter;
42         basdugum=temp;
43     }
44     return basdugum;
45 }
```

```
ornek8.cpp x
45 }
46 void listedekielemanlarigetir(node * basdugum){
47     if(basdugum==NULL){
48         cout<<"Listede getirilecek eleman yok";
49     }
50     node * temp=basdugum;
51     while(temp!=NULL){
52         cout<<temp->kullaniciadi<<" & "<<temp->parola<<endl;
53         temp=temp->next;
54     }
55 }
56 int main(){
57     node * root=NULL;
58     root=ekle(root,"ekle1",111);
59     root=ekle(root,"ekle2",222);
60     root=ekle(root,"ekle3",333);
61     root=ekle(root,"ekle4",444);
62     root=ekle(root,"ekle5",555);
63     listedekielemanlarigetir(root);
64     cout<<endl;
65     root=basdugumusil(root);
66     listedekielemanlarigetir(root);
67     cout<<endl;
68     root=basdugumusil(root);
69     listedekielemanlarigetir(root);
70     cout<<endl;
71     root=basdugumusil(root);
72     listedekielemanlarigetir(root);
73     cout<<endl;
74     root=basdugumusil(root);
75     listedekielemanlarigetir(root);
76     cout<<endl;
77     root=basdugumusil(root);
78     listedekielemanlarigetir(root);
79     cout<<endl;
80 }
```

Baştan Düğüm Silme Fonksiyonu

```
C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\tekyonlubagliliste>g++ -o ornek8 ornek8.cpp

C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\tekyonlubagliliste>ornek8
ekle1 & 111
ekle2 & 222
ekle3 & 333
ekle4 & 444
ekle5 & 555

ekle2 & 222
ekle3 & 333
ekle4 & 444
ekle5 & 555

ekle3 & 333
ekle4 & 444
ekle5 & 555

ekle4 & 444
ekle5 & 555

ekle5 & 555

Listede getirilecek eleman yok
```

Sondan Düğüm Silme Fonksiyonu

```
struct node * sondugumsil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Listede silinecek eleman yok";
    }
    else if(basdugum->next==NULL){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        struct node * iter=basdugum;
        while(iter->next->next!=NULL){
            iter=iter->next;
        }
        struct node * temp=iter->next;
        delete temp;
        iter->next=NULL;
    }
    return basdugum;
}
```

Sondan düğüm silmek için hedef metot yanda verilmiştir.

Somutlaştırılması ve kodlanması ders akışında ayrıntılı olarak yapılacaktır.


```
struct node * hedefdugumsil(node * basdugum,int
aranansifre){
    struct node * temp;
    struct node * iter=basdugum;
    if(basdugum->parola==aranansifre){
        temp=basdugum;
        basdugum=basdugum->next;
        delete temp;
        return basdugum;
    }
    while(iter->next->parola!=aranansifre){
        iter=iter->next;
    }
    if(iter->next==NULL){
        cout<<"Aranan sifre bulunamadi";
    }
    temp=iter->next;
    iter->next=temp->next;
    delete temp;
    return basdugum;
}
```

Hedef Düğümü Silme Fonksiyonu

Hedef düğümü silmek için hedef metot yanda verilmiştir.

Somutlaştırılması ve kodlanması ders akışında ayrıntılı olarak yapılacaktır.

A2 else{

```
node * iterkontrol=basdugum;
while(iterkontrol->next!=NULL &&
iterkontrol->next->parola!=hedefsifre){
    iterkontrol=iterkontrol->next;
}
if(iterkontrol->next==NULL){
    cout<<"Aranan eleman
bulunamamistir"<<endl;
}
node * iter=basdugum;
int adet=1;
while(iter->next->parola!=hedefsifre){
    adet++;
    iter=iter->next;
}
adet++;
node * temp=iter->next;
cout<<"hedef eleman"<<adet<<"inci
dugumde yer almaktadır"<<endl;

}

}
```

Hedef Düğümü Bulma Fonksiyonu

Hedef düğümü bulmak için
hedef metot yanda verilmiştir.

Somutlaştırılması ve kodlanması
ders akışında ayrıntılı olarak
yapılacaktır.

```
A1 void hedefdugumbul(node * basdugum,int hedefsifre){
    if(basdugum==NULL){
        cout<<"Listede eleman yok";
    }
    else if(basdugum->parola==hedefsifre){
        cout<<"Hedef dugum bulundu ve
bu dugum 1. dugumde yer almaktadır";
    }
}
```



```

void degistir(node * basdugum,int hedefsfre, string yenikullaniciadi){
    if(basdugum==NULL){
        cout<<"Listede eleman yok";
    }
    else if(basdugum->parola==hedefsfre){
        basdugum->kullaniciadi=yenikullaniciadi;
    }
    else{
        node * iterkontrol=basdugum;
        while(iterkontrol->next!=NULL && iterkontrol-
>next->parola!=hedefsfre){
            iterkontrol=iterkontrol->next;
        }
        if(iterkontrol->next==NULL){
            cout<<"Aranan eleman
bulunamamistir"<<endl;
        }
        node * iter=basdugum;
        while(iter->next->parola!=hedefsfre){
            iter=iter->next;
        }
        node * temp=iter->next;
        temp->kullaniciadi=yenikullaniciadi;
    }
}

```

Hedef Düğümün Kullanıcı Adını Değıştirme Fonksiyonu

Hedef Düğümün Kullanıcı Adını Değıştirme Fonksiyonu için hedef metot yanda verilmiştir.

Somutlaştırılması ve kodlanması ders akışında ayrıntılı olarak yapılacaktır.

```

cout<<"Kullanici adinin degismesini istediginiz sifreyi
giriniz"<<endl;cin>>hedefsfre;
cout<<"Hedef sifreye iliskin yeni kullanıcı adini
giriniz"<<endl;cin>>yenikullaniciadi;
degistir(root, hedefsfre,yenikullaniciadi);

```

Düğümlerdeki sayıların ortalamasını bulmak

```
void ortalama(node * basdugum){
    node * iter=basdugum;
    int toplam=0;
    int adet=0;
    while(iter!=NULL){
        toplam+=iter->parola;
        adet++;
        iter=iter->next;
    }
    cout<<"Listedeki yaklasik ortalama parola sonucu
    "<<toplam/adet<<" 'tir"<<endl;
}
```

Düğümlerdeki sayıların ortalamasını bulan fonksiyonun metodu yanda verilmiştir.

Somutlaştırılması ve kodlanması ders akışında ayrıntılı olarak yapılacaktır.

```
int main(){
    node * root=NULL;
    root=ekle(root,"ekle1",111);
    root=ekle(root,"ekle2",222);
    root=ekle(root,"ekle3",333);
    root=ekle(root,"ekle4",444);
    root=ekle(root,"ekle5",555);
    root=ekle(root,"ekle6",666);
    ortalama(root);
    return 0;
}
```

Düğümlerdeki sayıların minimumunu bulmak

```
int main(){
    node * root=NULL;
    root=ekle(root,"ekle1",777);
    root=ekle(root,"ekle2",555);
    root=ekle(root,"ekle3",111);
    root=ekle(root,"ekle4",110);
    root=ekle(root,"ekle5",999);
    root=ekle(root,"ekle6",666);
    root=ekle(root,"ekle6",111);
    root=ekle(root,"ekle6",111);
    mindeger(root);
    maxdeger(root);
    aynisay(root,111);
    return 0;
}
```

```
void mindeger(node * basdugum){
    if(basdugum==NULL){
        cout<<"liste bos"<<endl;
    }
    else{
        node * iter=basdugum;
        int tempsifre=iter->parola;
        while(iter->next!=NULL){
            iter=iter->next;
            if(iter->parola<tempsifre){
                tempsifre=iter->parola;
            }
        }

        cout<<"Listedeki minimum deger"<<tempsifre<<endl;
    }
}
```

Düğümlerdeki sayıların maksimumunu bulmak

```
int main(){
    node * root=NULL;
    root=ekle(root,"ekle1",777);
    root=ekle(root,"ekle2",555);
    root=ekle(root,"ekle3",111);
    root=ekle(root,"ekle4",110);
    root=ekle(root,"ekle5",999);
    root=ekle(root,"ekle6",666);
    root=ekle(root,"ekle6",111);
    root=ekle(root,"ekle6",111);
    mindeger(root);
    maxdeger(root);
    aynisay(root,111);
    return 0;
}
```

```
void maxdeger(node * basdugum){
    if(basdugum==NULL){
        cout<<"liste bos"<<endl;
    }
    else{
        node * iter=basdugum;
        int tempsifre=iter->parola;
        while(iter->next!=NULL){
            iter=iter->next;
            if(iter->parola>tempsifre){
                tempsifre=iter->parola;
            }
        }

        cout<<"Listedeki maksimum
deger"<<tempsifre<<endl;
    }
}
```

Düğümlerdeki aynı sayıların adetini bulmak

```
int main(){
    node * root=NULL;
    root=ekle(root,"ekle1",777);
    root=ekle(root,"ekle2",555);
    root=ekle(root,"ekle3",111);
    root=ekle(root,"ekle4",110);
    root=ekle(root,"ekle5",999);
    root=ekle(root,"ekle6",666);
    root=ekle(root,"ekle6",111);
    root=ekle(root,"ekle6",111);
    mindeger(root);
    maxdeger(root);
    aynisay(root,111);
    return 0;
}
```

```
void aynisay(node * basdugum,int hedef){
    if(basdugum==NULL){
        cout<<"Liste bos";
    }
    else{
        node * iter=basdugum;
        int tempsifre=iter->parola;
        int adet=0;
        while(iter->next!=NULL){
            if(iter->parola==hedef){
                adet++;
            }
            iter=iter->next;
        }
        if(iter->parola==hedef){
            adet++;
        }
        cout<<"Listede "<<hedef<<" elemana iliskin
        toplam "<<adet<<"veri vardır"<<endl;
    }
}
```

Dosyaya Yazmak ve Dosyadan Okumak

```
void dosyayayaz(node * basdugum){
    cout<<"Dosyaya Yaziliyor!!!";
    if(basdugum==NULL){
        cout<<"Eleman yok";
    }
    ofstream yaz("tekyonlubagliliste.txt",ios::app);
    struct node * iter=basdugum;
    while(iter!=NULL){
        yaz<<"Dugumun kullanıcı adı"<<iter-
>kullaniciadi<<"&"<<"Dugumun şifresi"<<iter->parola<<endl;
        iter=iter->next;
    }
    yaz.close();
}
```

ofstream → C++'ta dosya yazmak için kullanılan sınıf

"tekyonlubagliliste.txt" → yazacağınız dosyanın adı

ios::app → append modu: dosyanın sonuna ekleme yapar, var olan içerik silinmez

getline(oku, satir) → dosyadan satır satır okur

```
void dosyadanoku(){
    system("cls");
    ifstream oku("tekyonlubagliliste.txt");
    string satir;
    while(getline(oku,satir)){
        cout<<satir<<" "<<endl;
    }
    oku.close();
}

int main(){
    root=ekle(root,"ekle11",111);
    root=ekle(root,"ekle111",111);
    dosyayayaz(root);
    dosyadanoku();
    return 0;
}
```

burada string satir; tanımlaması çok önemli. Çünkü getline() fonksiyonu okuduğu satırı koyacağı bir değişken ister.

Dosyadan okunan her satır geçici olarak bu değişkende tutulur.

dosyadan satır oku → satir değişkenine koy → ekrana yaz → tekrar oku

KAYNAKÇA

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel, Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++ ve Java ile, Sadi Evren Şeker